7.2 Fast Poisson Solvers

The Poisson equation $-\partial^2 u/\partial x^2 - \partial^2 u/\partial y^2 = f(x,y)$ includes that source term $f(x,y)$. Like Laplace, it comes with boundary values on u or $\partial u/\partial n$, not initial values. This section 7.2 works on a square region with a square mesh ($\Delta x = \Delta y = h$). Finite differences and finite elements lead to equations $KU = F$, where the components $U_{i,j}$ are approximations to $u(i\Delta x, j\Delta y)$ at the N^2 meshpoints. *The goal is to solve* $KU = F$ *as quickly as possible.*

Poisson's equation is important in itself. Because its coefficients are constant and the region is a rectangle (in fact a square), Fourier methods can operate. There are quick and useful methods for this model problem—we explain **Fourier analysis and cyclic reduction**. Fast Poisson Solvers are valuable as preconditioners for nearby problems that have non-rectangular regions and non-constant coefficients. Two sources for software are . . . netlib, Numerical Recipes.

Start with the one-dimensional problem, where the idea is especially clear. The equation $-u'' = f(x)$ has boundary conditions $u(0) = u(1) = 0$. The interval $[0,1]$ is divided into $N+1$ pieces of length $h = \Delta x$. The difference equation for $U_1, \ldots, U_N$ involves the tridiagonal $-1, 2, -1$ second difference matrix K:

$$-U_{j+1} + 2U_j - U_{j-1} = h^2 f(jk) = F_j \qquad \text{with } U_0 = U_{N+1} = 0\,. \tag{1}$$

Express the input vector F as a combination of *discrete sines* $S_1, \ldots, S_N$. Then look for the solution vector U in the same form (the sines satisfy $U_0 = U_{N+1} = 0$):

$$\textbf{Sine transform} \qquad U_j = \sum_{k=1}^{N} d_k \sin\frac{\pi jk}{N+1} \qquad\qquad F_j = \sum_{k=1}^{N} b_k \sin\frac{\pi jk}{N+1} \tag{2}$$

Substitute those sums into equation (1). On the left side, the second difference of the discrete sine S_k is $\lambda_k S_k = (2 - 2\cos\frac{\pi k}{N+1})S_k$. These S_k are the *eigenvectors* of the matrix K, with eigenvalues λ_k as found in Section 1.6. Then we see each k separately:

$$\textbf{Each frequency} \qquad\qquad (2 - 2\cos\frac{\pi k}{N+1})d_k = h^2 b_k\,. \tag{3}$$

This determines d_k. The solution U is in (2). Finished.

Numerically, we compute the coefficients $b_1, \ldots, b_N$ (the Discrete Sine Transform of F). Equation (3) is just a division to find d_k. Then the sum in (2) gives U_j (the inverse transform). Those transforms are executed by a specialization of the FFT.

Algebraically, these steps diagonalized K into $S\Lambda S^{-1}$ by its sine eigenvectors. *The solution has three steps,* S^{-1} *then* Λ^{-1} *then* S. Compute the b's then the d's then U:

$$\textbf{Step 1} \quad Sb = F \qquad\quad \textbf{Step 2} \quad \Lambda d = b \qquad\quad \textbf{Step 3} \quad U = Sd \tag{4}$$

Practically, we wouldn't do this in one dimension! Ordinary elimination on a tridiagonal K needs only $O(N)$ operations. The sine transform (including the FFT) uses $O(N \log N)$. In two dimensions, the matrix order is N^2 *and its bandwidth is* N and ordinary elimination loses to the FFT (N^4 steps versus $N^2 \log N$). In three dimensions, elimination reaches a disastrous operation count of (order)(bandwidth)^2 = $(N^3)(N^2)^2 = N^7$.

Solving $KU = F$ by eigenvectors instead of elimination is very unusual. The eigenvector matrix S is full and normally hard to find (and costly to invert). This book has seen $K = A^{\mathrm{T}}CA$ and $K = LDL^{\mathrm{T}}$ more often than $K = S\Lambda S^{-1}$, but the sine matrix S is quick because of the Fast Fourier Transform.

May I also comment that those eigenvalues λ_k in equation (3) come easily from the $-1, 2, -1$ rows of K. **The quickest way is to substitute** $U_j = e^{\pi ijk/N+1}$:

$$-U_{j+1} + 2U_j - U_{j-1} = (-e^{\pi ik/N+1} + 2 - e^{-\pi ik/N+1})U_j = \lambda_k U_j \,. \tag{5}$$

That step to the same λ_k is easier for exponentials than for sines. The true eigenvectors are the sines, because they satisfy $U_0 = U_{N+1} = 0$. Other boundary conditions lead to cosines. The message of this chapter is right here: *Exponentials show quickly what is happening.*

Two-dimensional Model Problem

In two dimensions, the unit interval $[0, 1]$ becomes a unit square. The meshpoint $(x, y) = (ih, jh)$ is identified by two indices i and j, between 1 and N. The number of unknowns $U_{i,j}$ and equations $\mathbf{KU} = \mathbf{F}$ is now N^2. (Boldface indicates two dimensions.)

The equations come from second differences $-\Delta_x^2 - \Delta_y^2$ and also finite elements:

$$\mathbf{KU} = \mathbf{F} \qquad -U_{i+1,j} - U_{i-1,j} - U_{i,j+1} - U_{i,j-1} + 4U_{i,j} = F_{i,j} \,. \tag{6}$$

This is the *5-point scheme*. For finite differences, $F_{i,j}$ is usually the value of $h^2 f(x, y)$ at the i, j meshpoint. The finite elements are linear over each triangle, when we subdivide the mesh squares by 45° diagonal lines. The stiffness matrix from Section ____ is the same $\mathbf{K}$, and now the right side $\mathbf{F}$ involves integrals of $f(x, y)$. All we care about here is the *five-diagonal form* of the big matrix $\mathbf{K}$, and the *fast solution* of $\mathbf{KU} = \mathbf{F}$.

As always, the type of boundary condition decides the boundary rows of our matrix:

1. **Dirichlet condition** ($u = 0$ or $u = u_0(x, y)$ at the boundary)
2. **Neumann condition** (zero slope $\partial u/\partial n = 0$ at the boundary)
3. **Periodic conditions** (the problem repeats on neighboring squares)

Those match the special matrices K, T or B, and C at the start of this book.

Now the problem is two-dimensional. We stay with Dirichlet conditions $u = 0$, so that $U_{i,j} = 0$ when i or j is 0 or $N + 1$. **The 5-point matrix K is block**

tridiagonal:

$$\mathbf{K} = \begin{bmatrix} K+2I & -I & 0 & 0 & 0 \\ -I & K+2I & -I & 0 & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & 0 & -I & K+2I & -I \\ 0 & 0 & 0 & -I & K+2I \end{bmatrix}. \qquad (7)$$

The main diagonal has all 4's. There are four diagonals of -1's, with breaks at the end of blocks. A typical row has four -1's, but one is missing when (i,j) is next to the boundary (and we lose two next to a corner). Good thing, because if every row had four -1's, then $\mathbf{K}$ would be singular.

The N^2 eigenvectors of $\mathbf{K}$ are *separable*, and they separate into products of sines:

Eigenvectors $\mathbf{S}_{kl}$ of K The (i,j) component of $\mathbf{S}_{kl}$ is $\sin\dfrac{\pi ik}{N+1}\sin\dfrac{\pi jl}{N+1}$.(8)

When you multiply that eigenvector by $\mathbf{K}$, you have second differences of the first sine (in the x-direction) plus second differences of the second sine (in the y-direction). So the eigenvalue in two dimensions is the sum $\lambda_k + \lambda_l$ of one-dimensional eigenvalues:

Eigenvalues $\mathbf{KS}_{kl} = \lambda_{kl}\mathbf{S}_{kl}$ $\qquad \lambda_{kl} = (2 - 2\cos\dfrac{\pi k}{N+1}) + (2 - 2\cos\dfrac{\pi l}{N+1})$. (9)

We get this same λ_{kl} from second differences of exponentials (Problem ____).

Now the solution of $\mathbf{KU} = \mathbf{F}$ comes by *two-dimensional sine transform*:

$$U_{i,j} = \sum\sum d_{kl}\sin\frac{\pi ik}{N+1}\sin\frac{\pi jl}{N+1} \qquad F_{i,j} = \sum\sum b_{kl}\sin\frac{\pi ik}{N+1}\sin\frac{\pi jl}{N+1} \qquad (10)$$

Substituting into the five-point equation (6), which is $\mathbf{KU} = \mathbf{F}$, gives $\lambda_{kl}d_{kl} = b_{kl}$. So again we compute the b's, divide by the λ's, and construct $\mathbf{U}$ from the d's:

Step 1 $\mathbf{Sb} = \mathbf{F}$ $\qquad$ **Step 2** $d_{kl} = \dfrac{b_{kl}}{\lambda_{kl}}$ $\qquad$ **Step 3** $\mathbf{U} = \mathbf{Sd}$

Swartztrauber [SIAM Review **19** (1977) 490] gives the operation count $2N^2\log_2 N$.

Cyclic Reduction

There is an entirely different (and very simple) approach to $\mathbf{KU} = \mathbf{F}$. I will start in one dimension, by writing down three rows of the familiar equation $KU = F$:

$$\begin{array}{lrcl} \text{Row } i-1 & -U_{i-2} + 2U_{i-1} - U_i & & = F_{i-1} \\ \text{Row } i & -U_{i-1} + 2U_i - U_{i+1} & & = F_i \\ \text{Row } i+1 & -U_i + 2U_{i+1} - U_{i+2} & & = F_{i+1} \end{array} \qquad (11)$$

Multiply the middle equation by 2, and add. *This eliminates U_{i-1} and U_{i+1}*:

$$\textbf{Reduction} \qquad -U_{i-2} + 2U_i - U_{i+2} = f_{i-1} + 2f_i + f_{i+1}\,. \tag{12}$$

Now we have a *half-size system.* It involves only half of the U's (with even indices), and in fact (12) has the same tridiagonal form as before. When we repeat, *cyclic reduction produces a quarter-size system.* Eventually we can reduce $KU = F$ to a very small problem, and then cyclic back-substitution produces the whole solution.

How does this look in two dimensions? The big matrix $\mathbf{K}$ in (7) has the block form

$$\mathbf{K} = \begin{bmatrix} A & -I & & \\ -I & A & -I & \\ & \cdot & \cdot & \cdot \\ & & -I & A \end{bmatrix} \qquad \text{with} \quad A = K + 2I\,. \tag{13}$$

The three equations in (11) become *block equations for whole rows of N mesh values.* We are taking the unknowns $\mathbf{U}_i = (U_{i1}, \ldots, U_{iN})$ a row at a time. If we write three rows of (13), the block A replaces the number 2 in the scalar equation (11). The block $-I$ replaces the number -1. To reduce $\mathbf{KU} = \mathbf{F}$ to a half-size system, multiply the middle equation (with i even) by A and add the three block equations:

$$\textbf{Reduction} \qquad -I\mathbf{U}_{i-2} + (A^2 - 2I)\mathbf{U}_i - I\mathbf{U}_{i+2} = \mathbf{F}_{i-1} + A\mathbf{F}_i + \mathbf{F}_{i+1}\,. \tag{14}$$

The new half-size matrix is still block tridiagonal. *The diagonal blocks that were previously A are now $A^2 - 2I$*, with the same eigenvectors. The unknowns are the $\frac{1}{2}N^2$ values $U_{i,j}$ at meshpoints with even indices i.

The bad point is that the new diagonal blocks $A^2 - 2I$ have *five diagonals*, where the original blocks $A = K + 2I$ had three diagonals. This bad point gets worse as cyclic reduction continues. At step r, the diagonal blocks become $A_r = A_{r-1}^2 - 2I$ and the bandwidth doubles.

We could find tridiagonal factors $(A - \sqrt{2}I)(A + \sqrt{2}I) = A^2 - 2I$, but the number of factors grows quickly. Storage and computation and roundoff error are increasing too rapidly with more reduction steps.

Stable variants of cyclic reduction were developed by Buneman and Hockney. The clear explanation by Buzbee, Golub, and Nielson [SIAM Journal of Numerical Analysis **7** (1970) 627] allows other boundary conditions and other separable equations, coming from polar coordinates in a circle or from convection terms like $C\partial u/\partial x + D\partial u/\partial y$. After only l steps of cyclic reduction, Hockney applied Fourier analysis (he went back to eigenvectors).

This combination FACR(l) is widely used and the optimal number of reduction steps is small. For $N = 128$ a frequent choice is $l = 2$. Asymptotically $l_{\mathbf{opt}}$ grows like $\log\log N$ and the operation count for FACR($l_{\mathbf{opt}}$) is $3N^2 \log\log N$. In practical scientific computing with N^2 unknowns (or with N^3 unknowns in three dimensions), a Fast Poisson solver is a winner.